

Nguyên Lý Thiết Kế Phần Mềm

Design Principles

Design Principles

Mục tiêu buổi học

- ☕ Hiểu và áp dụng các nguyên tắc thiết kế cơ bản
- ☕ Nắm vững nguyên lý **SOLID**
- ☕ Biết cách tạo ra mã nguồn linh hoạt, dễ bảo trì

Nội dung chính

- 1 Coupling & Cohesion
- 2 Nguyên lý **SOLID, DRY, KISS, YAGNI**

WHY

Vấn đề của mã nguồn thiết kế kém

- ☕ **Khó bảo trì:** Sửa một chỗ ảnh hưởng nhiều nơi
- ☕ **Khó mở rộng:** Thêm tính năng mới phải sửa code cũ
- ☕ **Khó test:** Code phụ thuộc chặt chẽ vào nhau
- ☕ **Khó hiểu:** Logic phức tạp, chức năng lộn xộn

Lợi ích của thiết kế tốt

- ☕ Dễ đọc và hiểu
- ☕ Dễ sửa đổi và mở rộng
- ☕ Dễ kiểm thử
- ☕ Tái sử dụng cao

COUPLING & COHESION

Cohesion

Cohesion là mức độ các thành phần trong một module/class làm việc cùng nhau để thực hiện một mục đích rõ ràng.

Phân loại:

- ☕ **High Cohesion** (Tốt): Các phương thức/thuộc tính liên quan chặt chẽ
- ☕ **Low Cohesion** (Xấu): Các phương thức/thuộc tính không liên quan

Low Cohesion

Vấn đề: Class Employee làm quá nhiều việc: quản lý dữ liệu, in ấn, database, email.

```
// Class làm quá nhiều việc không liên quan
public class Employee {
    private String name;
    private double salary;

    public void calculateSalary() { } // Nghiệp vụ nhân viên
    public void printReport() { } // In báo cáo
    public void saveToDatabase() { } // Lưu database
    public void sendEmail() { } // Gửi email
}
```


High Cohesion

Lợi ích: Mỗi class có một trách nhiệm rõ ràng, dễ test và bảo trì.

```
// Class chỉ quản lý dữ liệu nhân viên
public class Employee {
    private String name, double salary;
    public String getName() { return name; }
    public double getSalary() { return salary; }
    public void calculateSalary() { }
}
```

```
// Class riêng cho in báo cáo
public class ReportPrinter {
    public void printEmployeeReport(Employee emp) { }
}

// Class riêng cho database
public class EmployeeRepository {
    public void save(Employee emp) { }
}

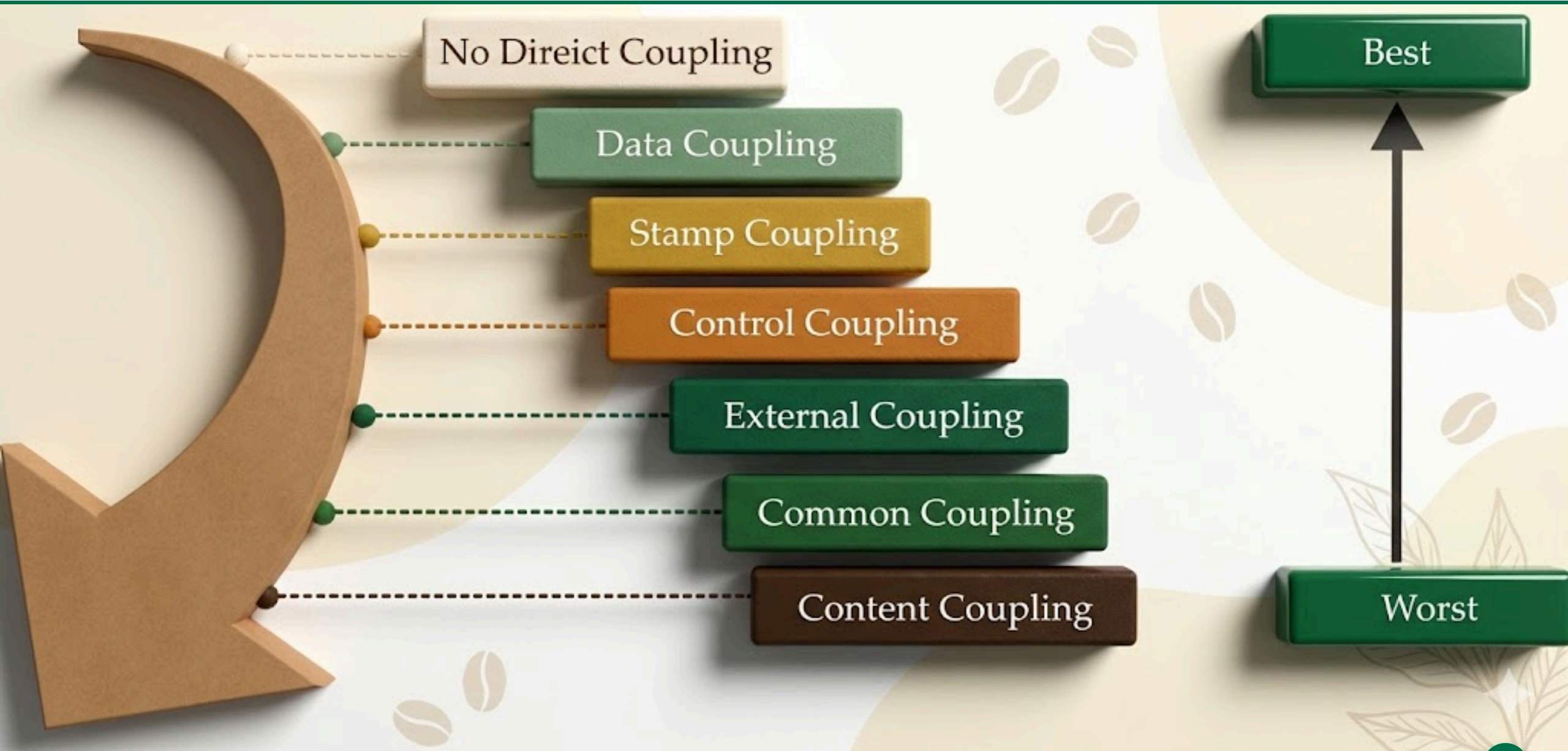
// Class riêng cho email
public class EmailService {
    public void sendToEmployee(Employee emp, String message) { }
}
```

Coupling

Coupling là mức độ phụ thuộc giữa các `module/class` với nhau.

Phân loại

- ☕ **Loose Coupling:** Ít phụ thuộc, thay đổi một module ít ảnh hưởng module khác 👍
- ☕ **Tight Coupling:** Phụ thuộc chặt, thay đổi một module ảnh hưởng nhiều module 👎



Tight Coupling

```
// OrderProcessor phụ thuộc chặt vào MySQLDatabase
public class OrderProcessor {
    private MySQLDatabase database;

    public OrderProcessor() {
        // Khởi tạo cứng - TIGHT COUPLING
        this.database = new MySQLDatabase();
    }

    public void processOrder(Order order) {
        // Logic xử lý
        database.save(order);
    }
}
```

Vấn đề

- ☕ Không thể đổi sang **PostgreSQL/MongoDB**
- ☕ Khó test vì phụ thuộc vào database thật
- ☕ Thay đổi **MySQLDatabase** ảnh hưởng **OrderProcessor**

Loose Coupling

```
public interface Database {  
    void save(Order order);  
}  
  
public class MySQLDatabase implements Database {  
    public void save(Order order) { /* MySQL implementation */ }  
}  
  
public class MongoDBDatabase implements Database {  
    public void save(Order order) { /* MongoDB implementation */ }  
}  
  
// OrderProcessor phụ thuộc vào interface  
public class OrderProcessor {  
    private Database database;  
  
    // Dependency Injection  
    public OrderProcessor(Database database) {  
        this.database = database;  
    }  
  
    public void processOrder(Order order) {  
        database.save(order);  
    }  
}
```

Lợi ích

- ☕ Dễ dàng thay đổi MySQL → MongoDB
- ☕ Dễ test với mock/fake database
- ☕ Thay đổi **MySQLDatabase** không ảnh hưởng **OrderProcessor**

Golden Goal

High Cohesion + Low Coupling = Maintainable Code

SOLID PRINCIPLES

Tổng Quan

-  **S** - Single Responsibility Principle
-  **O** - Open/Closed Principle
-  **L** - Liskov Substitution Principle
-  **I** - Interface Segregation Principle
-  **D** - Dependency Inversion Principle

Mục đích

- ☕ Làm cho phần mềm dễ hiểu, linh hoạt và bảo trì
- ☕ Giảm sự phức tạp khi thay đổi
- ☕ Tăng khả năng tái sử dụng

Single Responsibility Principle (SRP)

Một class chỉ nên có một lý do để thay đổi

Giải thích

- ☕ Mỗi class chỉ làm một việc duy nhất
- ☕ Chỉ một **actor** (người dùng/stakeholder) có thể yêu cầu thay đổi `class`
- ☕ Tránh **God Class** - class làm mọi việc

Eg: Violate SRP

Vấn đề: `User` class có 5 lý do để thay đổi!

```
public class User {  
    private String name;  
    private String email;  
  
    // Nghiệp vụ User  
    public void updateProfile(String name, String email) {  
        this.name = name;  
        this.email = email;  
    }  
  
    // Validation - Lý do thay đổi #2  
    public boolean validateEmail() {  
        return email.contains("@");  
    }  
  
    // Database - Lý do thay đổi #3  
    public void saveToDatabase() { ... }  
}
```

```
// Email - Lý do thay đổi #4  
public void sendWelcomeEmail() {  
    // Email sending code  
}  
  
// Report - Lý do thay đổi #5  
public String generateReport() {  
    return "User: " + name;  
}  
}
```

Eg: Follow SRP

```
// 1. Chỉ quản lý dữ liệu user
public class User {
    private String name;
    private String email;

    public void updateProfile(String name, String email) {
        this.name = name;
        this.email = email;
    }

    public String getName() { return name; }
    public String getEmail() { return email; }
}

// 2. Chỉ validate
public class UserValidator {
    public boolean validateEmail(String email) {
        return email.contains("@") && email.contains(".");
    }
}
```

```
// 3. Chỉ xử lý database
public class UserRepository {
    public void save(User user) { /* DB code */ }
    public User findById(String id) { /* DB code */ return null; }
}

// 4. Chỉ gửi email
public class EmailService {
    public void sendWelcomeEmail(User user) { /* Email code */ }
}

// 5. Chỉ tạo báo cáo
public class UserReportGenerator {
    public String generate(User user) {
        return "User Report: " + user.getName();
    }
}
```

Advantages of SRP

- ☕ **Dễ hiểu:** Mỗi class có một mục đích rõ ràng
- ☕ **Dễ test:** Test từng trách nhiệm riêng biệt
- ☕ **Dễ bảo trì:** Thay đổi validation không ảnh hưởng database
- ☕ **Tái sử dụng:** `EmailService` có thể dùng cho `Order`, `Product` ...
- ☕ **Giảm conflict:** Team có thể làm việc song song

Câu hỏi kiểm tra: Class này có bao nhiêu lý do để thay đổi?"

Open/Closed Principle (OCP)

Giải thích

- ☕ **Open for extension:** Có thể thêm tính năng mới
- ☕ **Closed for modification:** Không sửa code cũ khi thêm tính năng

Phương pháp

- ☕ Sử dụng abstraction (**interface**, **abstract class**)
- ☕ Thêm tính năng bằng cách *tạo class mới*
- ☕ *Không sửa code* đã test và hoạt động tốt

Eg: Violate OCP

```
public class PaymentProcessor {  
    public void processPayment(String type, double amount) {  
        if (type.equals("CREDIT_CARD")) {  
            System.out.println("Processing credit card: " + amount);  
            // Credit card logic  
        }  
        else if (type.equals("PAYPAL")) {  
            System.out.println("Processing PayPal: " + amount);  
            // PayPal logic  
        }  
        // Thêm Momo → phải SỬA code này!  
        else if (type.equals("MOMO")) {  
            System.out.println("Processing Momo: " + amount);  
            // Momo logic  
        }  
    }  
}
```

Phân tích

- ☕ Mỗi lần thêm phương thức thanh toán mới phải sửa `PaymentProcessor`
- ☕ Phải test lại toàn bộ code -> Rủi ro phá vỡ code cũ

Eg: Follow OCP

```
// Interface - abstraction
public interface PaymentMethod {
    void process(double amount);
}

// Implementation 1
public class CreditCardPayment implements PaymentMethod {
    public void process(double amount) {
        System.out.println("Processing credit card: " + amount);
        // Credit card logic
    }
}

// Implementation 2
public class PayPalPayment implements PaymentMethod {
    public void process(double amount) {
        System.out.println("Processing PayPal: " + amount);
        // PayPal logic
    }
}
```

```
// Implementation 3 - THÊM MỚI mà KHÔNG SỬA code cũ
public class MomoPayment implements PaymentMethod {
    public void process(double amount) {
        System.out.println("Processing Momo: " + amount);
        // Momo logic
    }
}

// Processor không cần sửa
public class PaymentProcessor {
    public void processPayment(PaymentMethod method, double amount) {
        method.process(amount);
    }
}
```

Eg: Follow OCP (cont)

```
public class Main {  
    public static void main(String[] args) {  
        PaymentProcessor processor = new PaymentProcessor();  
  
        // Sử dụng Credit Card  
        processor.processPayment(new CreditCardPayment(), 100.0);  
  
        // Sử dụng PayPal  
        processor.processPayment(new PayPalPayment(), 200.0);  
  
        // Sử dụng Momo - KHÔNG CẦN SỬA PaymentProcessor  
        processor.processPayment(new MomoPayment(), 150.0);  
    }  
}
```

Lợi ích

- ☕ Thêm phương thức thanh toán mới -> tạo class mới
- ☕ Không sửa code đã test
- ☕ Không rủi ro phá vỡ code cũ
- ☕ Dễ maintain và mở rộng

Liskov Substitution Principle (LSP)

Nguyên lý Thay Thế Liskov: Các đối tượng của lớp con phải có thể thay thế được các đối tượng của lớp cha mà không làm thay đổi tính đúng đắn của chương trình

Giải thích

- ☕ Nếu **S** là subtype của **T**, thì có thể thay **T** bằng **S** mà không gây lỗi
- ☕ Subclass phải tuân thủ **hợp đồng** (contract) của superclass
- ☕ Không được thay đổi hành vi mong đợi

```
Parent p = new Child(); // Phải hoạt động đúng
```

Eg: Violate LSP

```
public class Rectangle {  
    protected int width;  
    protected int height;  
  
    public void setWidth(int width) {  
        this.width = width;  
    }  
  
    public void setHeight(int height) {  
        this.height = height;  
    }  
  
    public int getArea() {  
        return width * height;  
    }  
}
```

```
public class Square extends Rectangle {  
    @Override  
    public void setWidth(int width) {  
        this.width = width;  
        this.height = width; // Vi phạm kỳ vọng!  
    }  
  
    @Override  
    public void setHeight(int height) {  
        this.width = height; // Vi phạm kỳ vọng!  
        this.height = height;  
    }  
}
```

Eg: Violate LSP (cont)

```
public class Test {  
    public static void main(String[] args) {  
        Rectangle rect = new Square(); // Thay thế  
        rect.setWidth(5);  
        rect.setHeight(4);  
  
        // Kỳ vọng: 5 * 4 = 20  
        // Thực tế: 4 * 4 = 16 (vì Square thay đổi cả width!)  
        System.out.println(rect.getArea()); // 16 ???  
    }  
}
```

Vấn đề:

- ☕ **Square** thay đổi hành vi của **Rectangle**
- ☕ Không thể thay thế **Rectangle** bằng **Square**
- ☕ Vi phạm **hợp đồng** của **Rectangle**

Eg: Follow LSP

```
// Interface chung
public interface Shape {
    int getArea();
}

// Rectangle - độc lập
public class Rectangle implements Shape {
    private int width;
    private int height;

    public Rectangle(int width, int height) {
        this.width = width;
        this.height = height;
    }
}
```

```
public int getArea() {
    return width * height;
}

// Square - độc lập
public class Square implements Shape {
    private int side;

    public Square(int side) {
        this.side = side;
    }

    public int getArea() {
        return side * side;
    }
}
```


Interface Segregation Principle (ISP)

Nguyên lý Phân Tách Interface: Không nên ép **client** phụ thuộc vào các **interface** mà chúng *không sử dụng*

Giải thích

- ☕ Tách interface lớn thành nhiều interface nhỏ, chuyên biệt
- ☕ Client chỉ implement những gì cần thiết
- ☕ Tránh **fat interface** - interface có quá nhiều method
 - 🌿 Chuyển thành nhiều **focused interface**

Eg: Violate ISP

Vấn đề: `RobotWorker` bị ép implement `eat()` và `sleep()` không cần thiết.

```
// Fat Interface - quá nhiều method
public interface Worker {
    void work();
    void eat();
    void sleep();
    void getPaid();
}

// Human worker - OK
public class HumanWorker implements Worker {
    public void work() { System.out.println("Working..."); }
    public void eat() { System.out.println("Eating..."); }
    public void sleep() { System.out.println("Sleeping..."); }
    public void getPaid() { System.out.println("Getting paid..."); }
}
```

```
// Robot worker - VI PHẠM!
public class RobotWorker implements Worker {
    public void work() { System.out.println("Working..."); }

    // Robot không ăn/ngủ nhưng BẮT BUỘC phải implement!
    public void eat() {
        throw new UnsupportedOperationException("Robot doesn't eat");
    }
    public void sleep() {
        throw new UnsupportedOperationException("Robot doesn't sleep");
    }

    public void getPaid() { System.out.println("Getting charged..."); }
}
```

Follow ISP

```
// Tách thành nhiều interface nhỏ
public interface Workable {
    void work();
}

public interface Eatable {
    void eat();
}

public interface Sleepable {
    void sleep();
}

public interface Payable {
    void getPaid();
}
```

```
// Human implement tất cả
public class HumanWorker implements Workable, Eatable, Sleepable, Payable {
    public void work() { System.out.println("Working..."); }
    public void eat() { System.out.println("Eating..."); }
    public void sleep() { System.out.println("Sleeping..."); }
    public void getPaid() { System.out.println("Getting paid..."); }
}

// Robot chỉ implement những gì cần
public class RobotWorker implements Workable, Payable {
    public void work() { System.out.println("Working..."); }
    public void getPaid() { System.out.println("Getting charged..."); }
    // Không cần eat(), sleep()!
}
```

Advantages of ISP

- ☕ **Linh hoạt:** Class chỉ implement những gì cần
- ☕ **Dễ hiểu:** Interface nhỏ, mục đích rõ ràng
- ☕ **Giảm coupling:** Thay đổi interface ít ảnh hưởng
- ☕ **Dễ test:** Mock/stub dễ dàng hơn

Realworld Example

```
// DAO pattern
public interface ReadableRepository<T> {
    T findById(String id);
    List<T> findAll();
}

public interface WritableRepository<T> {
    void save(T entity);
    void delete(String id);
}

// Read-only service chỉ cần ReadableRepository
public class ReportService {
    private ReadableRepository<Order> orderRepo;
    // Không thể gọi save() hay delete()
}
```

Dependency Inversion Principle (DIP)

Nguyên lý Đảo Ngược Sự Phụ Thuộc

- ☕ *High-level modules* không nên phụ thuộc vào *low-level modules*. Cả hai nên phụ thuộc vào abstraction.
- ☕ *Abstraction* không nên phụ thuộc vào details. *Details* nên phụ thuộc vào abstraction.

Giải thích

- ☕ Phụ thuộc vào **interface/abstract class**, không phụ thuộc vào concrete class
- ☕ Đảo ngược hướng phụ thuộc: từ high → low thành cả hai → abstraction

Eg: Violate DIP

```
// Low-level module
public class MySQLDatabase {
    public void save(String data) {
        System.out.println("Saving to MySQL: " + data);
    }
}

// High-level module PHỤ THUỘC vào low-level
public class UserService {
    private MySQLDatabase database; // Phụ thuộc concrete class!

    public UserService() {
        this.database = new MySQLDatabase(); // Tight coupling!
    }

    public void createUser(String userData) {
        // Business logic
        database.save(userData);
    }
}
```

Vấn đề

-  **UserService** phụ thuộc trực tiếp vào **MySQLDatabase**
-  Không thể đổi sang PostgreSQL, MongoDB
-  Khó test vì phải có database MySQL thật

Eg: Follow DIP

```
// Abstraction
public interface Database {
    void save(String data);
}

// Low-level module phụ thuộc vào abstraction
public class MySQLDatabase implements Database {
    public void save(String data) {
        System.out.println("Saving to MySQL: " + data);
    }
}

public class PostgreSQLDatabase implements Database {
    public void save(String data) {
        System.out.println("Saving to PostgreSQL: " + data);
    }
}
```

```
// High-level module phụ thuộc vào abstraction
public class UserService {
    private Database database; // Phụ thuộc interface!

    // Dependency Injection
    public UserService(Database database) {
        this.database = database;
    }

    public void createUser(String userData) {
        // Business logic
        database.save(userData);
    }
}
```

Eg: Usage of DIP

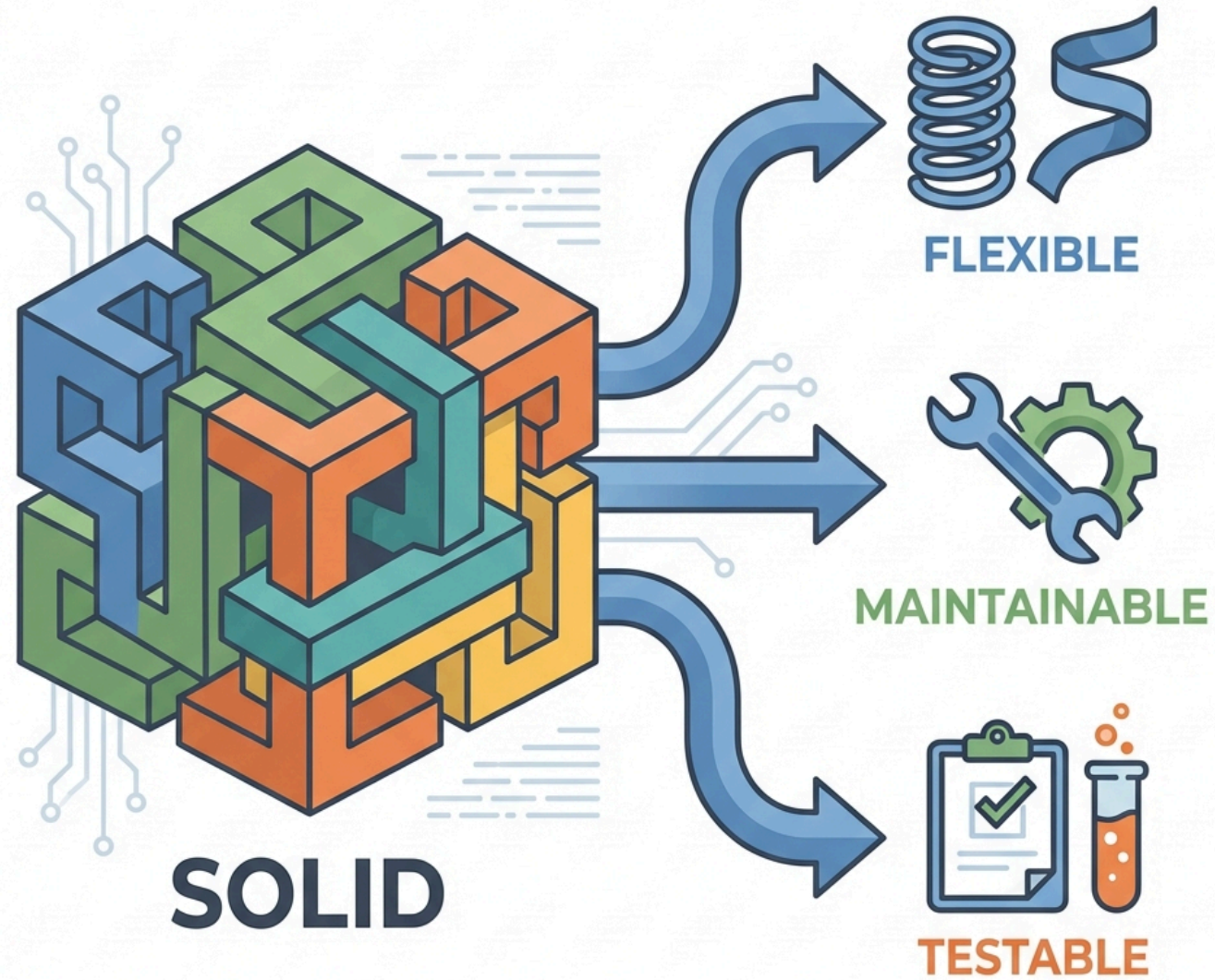
```
public class Main {  
    public static void main(String[] args) {  
        // Inject MySQL  
        Database mysqlDb = new MySQLDatabase();  
        UserService userService1 = new UserService(mysqlDb);  
        userService1.createUser("John Doe");  
  
        // Đổi sang PostgreSQL - KHÔNG SỬA UserService  
        Database postgresDb = new PostgreSQLDatabase();  
        UserService userService2 = new UserService(postgresDb);  
        userService2.createUser("Jane Doe");  
  
        // Test với Mock  
        Database mockDb = new MockDatabase();  
        UserService testService = new UserService(mockDb);  
        testService.createUser("Test User");  
    }  
}
```

Lợi ích

- ☕ Dễ thay đổi implementation
- ☕ Dễ test với **mock/stub**
- ☕ Giảm coupling
- ☕ Tăng tính linh hoạt

Summary

Nguyên lý	Ý nghĩa	Cách nhớ
S RP	1 class = 1 trách nhiệm	Một việc thôi!
O CP	Mở rộng, không sửa	Thêm mới, không đụng cũ
L SP	Subclass thay thế được Parent	Con thay cha được
I SP	Interface nhỏ, chuyên biệt	Không ép làm việc không cần
D IP	Phụ thuộc vào abstraction	Phụ thuộc interface , không concrete



CÁC NGUYÊN LÝ KHÁC

DRY - Don't Repeat Yourself

Every piece of knowledge must have a single, unambiguous, authoritative representation within a system

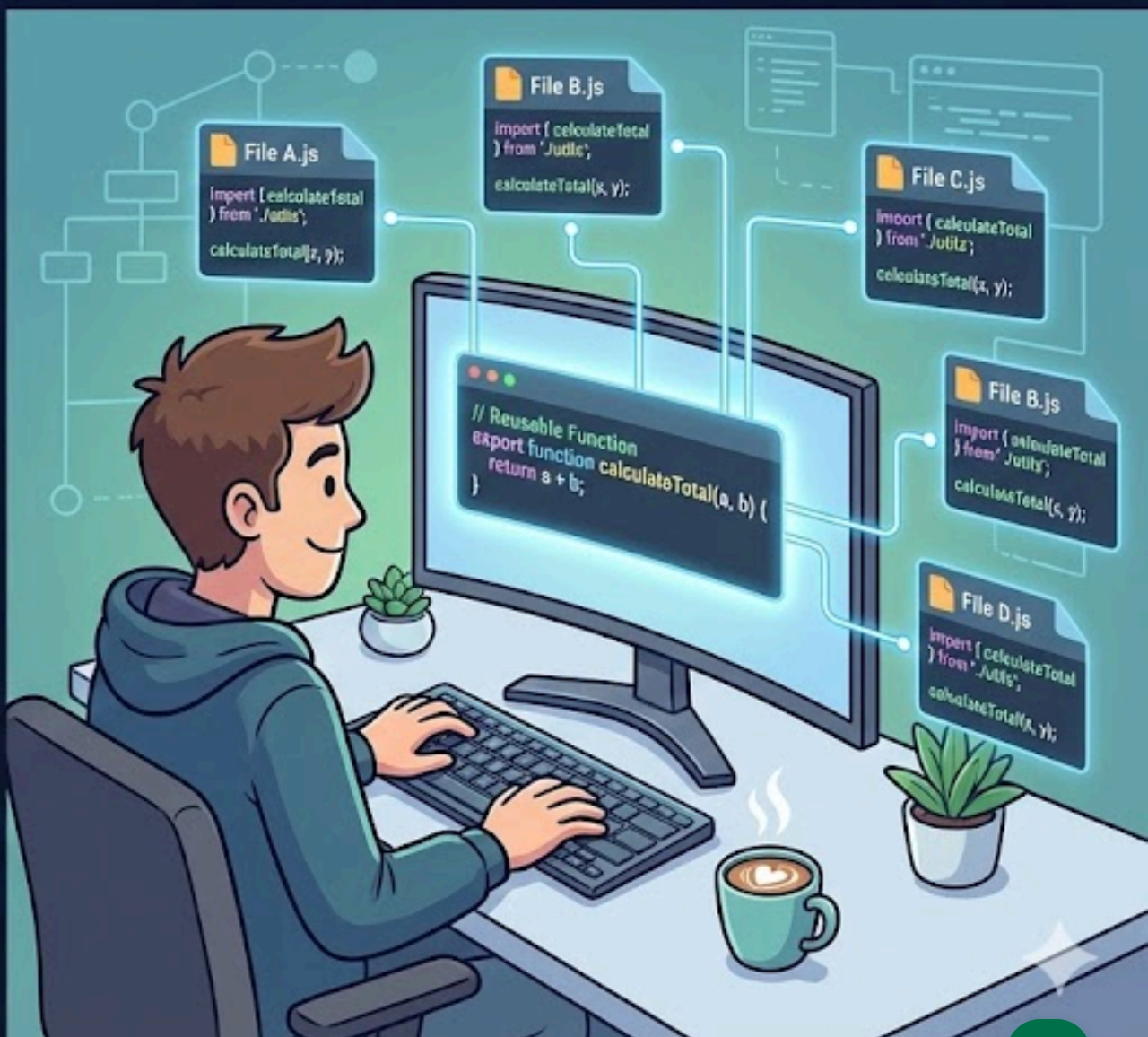
Giải thích

- ☕ Mỗi đoạn logic chỉ nên xuất hiện một lần
- ☕ Tránh copy-paste code
- ☕ Tái sử dụng thông qua `function`, `class`, `module`

DON'T - Repetitive Code



DO - DRY Principle: Write Once, Use Everywhere



Eg: Violate DRY (WET - Write Everything Twice)

Vấn đề: Logic validate bị lặp lại → Sửa ở một chỗ, quên chỗ kia!

```
public class OrderService {  
    public void createOrder(Order order) {  
        // Validate  
        if (order.getItems().isEmpty()) {  
            throw new IllegalArgumentException("Order must have items");  
        }  
        if (order.getCustomer() == null) {  
            throw new IllegalArgumentException("Order must have customer");  
        }  
  
        // Save  
        database.save(order);  
    }  
}
```

```
public void updateOrder(Order order) {  
    // DUPLICATE validation logic  
  
    if (order.getItems().isEmpty()) {  
        throw new IllegalArgumentException("Order must have items");  
    }  
    if (order.getCustomer() == null) {  
        throw new IllegalArgumentException("Order must have customer");  
    }  
  
    // Update  
    database.update(order);  
}
```

Follow DRY

```
public class OrderService {  
    // Extract validation logic  
    private void validateOrder(Order order) {  
        if (order.getItems().isEmpty()) {  
            throw new IllegalArgumentException("Order must have items");  
        }  
        if (order.getCustomer() == null) {  
            throw new IllegalArgumentException("Order must have customer");  
        }  
    }  
  
    public void createOrder(Order order) {  
        validateOrder(order); // Reuse  
        database.save(order);  
    }  
  
    public void updateOrder(Order order) {  
        validateOrder(order); // Reuse  
        database.update(order);  
    }  
}
```

Lợi ích

-  Sửa một chỗ, ảnh hưởng toàn bộ
-  Dễ maintain
-  Giảm bug
-  Code ngắn gọn hơn

DRY - Should

-  Logic nghiệp vụ giống nhau
-  Validation rules
-  Tính toán phức tạp
-  Database queries

DRY - Shouldn't

Duplication is better than *wrong abstraction*

- ☕ Code giống nhau nhưng khác ý nghĩa nghiệp vụ
- ☕ Code đơn giản (1-2 dòng)
- ☕ Logic có thể thay đổi độc lập

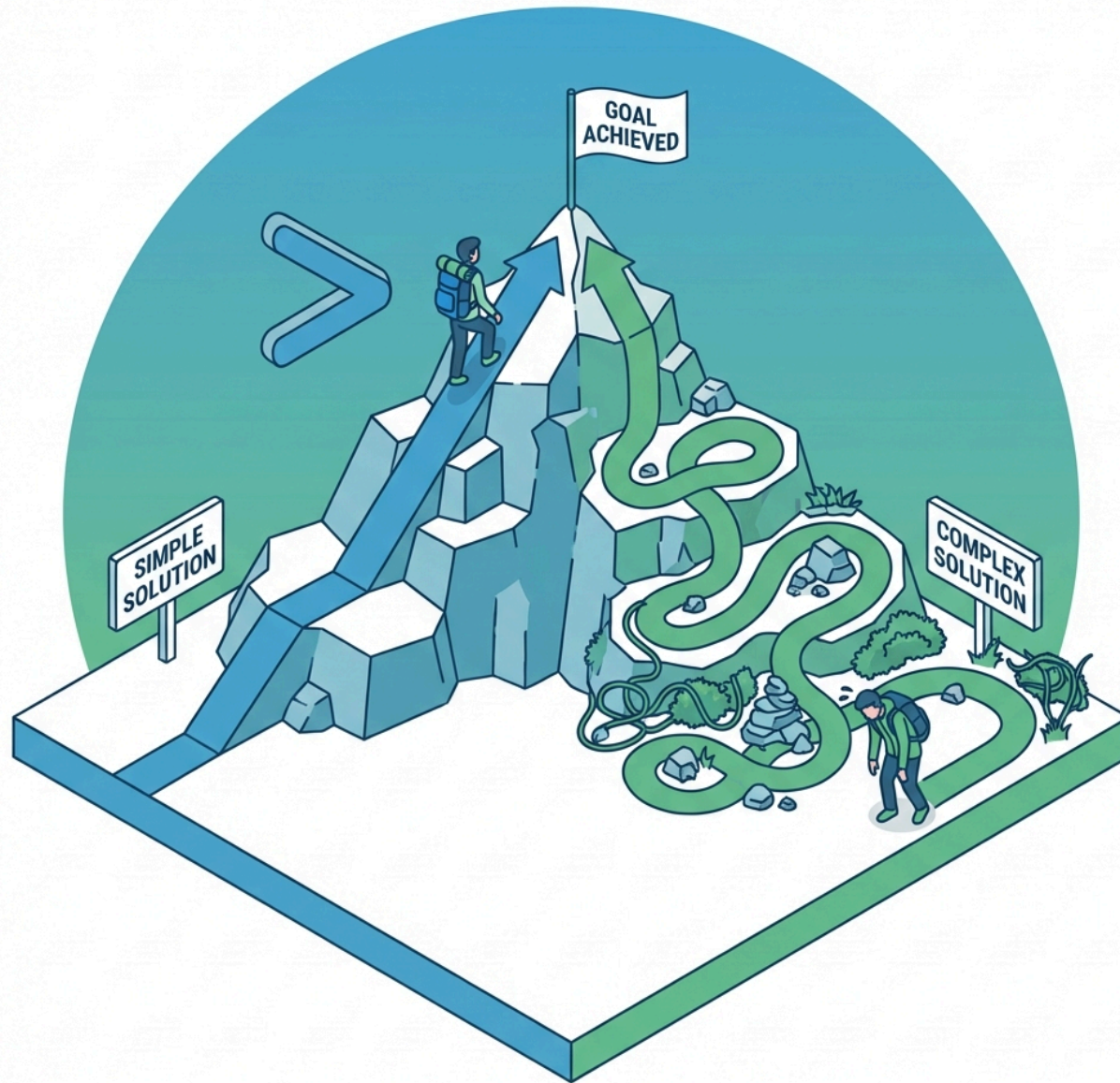
```
// Hai method này GIỐNG NHAU nhưng khác ý nghĩa nghiệp vụ
public double calculateOrderTotal() { return price * quantity; }
public double calculateTax() { return price * quantity; } // Tính cờ giống
```


KISS - Keep It Simple, Stupid

Simplicity should be a key goal in design, and *unnecessary complexity* should be avoided

Giải thích

- ☕ Giải pháp đơn giản nhất thường là tốt nhất
- ☕ Tránh **over-engineering**
- ☕ Code dễ đọc > Code *thông-minh*



Eg: Violate KISS (Over-engineer)

```
// Cách phức tạp không cần thiết
public class NumberChecker {
    public boolean isEven(int number) {
        // Sử dụng bitwise operation "cho pro"
        return (number & 1) == 0;
    }

    public String getStatus(boolean active) {
        // Ternary lồng nhau
        return active ? (active == true ? "ACTIVE" : "INACTIVE") : "INACTIVE";
    }

    public int sum(int[] numbers) {
        // Stream phức tạp cho việc đơn giản
        return Arrays.stream(numbers)
            .boxed()
            .collect(Collectors
                .summingInt(Integer::intValue));
    }
}
```

Vấn đề

-  Khó đọc, khó hiểu
-  Không cần thiết phức tạp
-  Dễ gây bug

Eg: Follow KISS

```
// Giải pháp đơn giản, dễ hiểu
public class NumberChecker {
    public boolean isEven(int number) {
        return number % 2 == 0; // Đơn giản
    }

    public String getStatus(boolean active) {
        return active ? "ACTIVE" : "INACTIVE"; // Rõ ràng
    }

    public int sum(int[] numbers) { // Đơn giản nhất
        int total = 0;
        for (int num : numbers) {
            total += num;
        }
        return total;
    }
}
```

Lợi ích

- ☕ Dễ đọc, dễ hiểu, kể cả với *sinh viên năm 1*
- ☕ Dễ maintain
- ☕ Ít bug hơn

KISS - HOW

Người khác có hiểu code này không?

Tránh premature optimization:

```
// Không cần: Optimize trước khi có vấn đề  
// Nên: Code đơn giản trước, optimize sau khi đo đạc
```

Sử dụng tên biến rõ ràng:

```
// Xấu: int d;  
// Tốt: int daysSinceCreation;
```


KISS - HOW

Giữ function ngắn gọn:

```
// Mỗi function làm 1 việc, < 20 dòng
```

Tránh deep nesting:

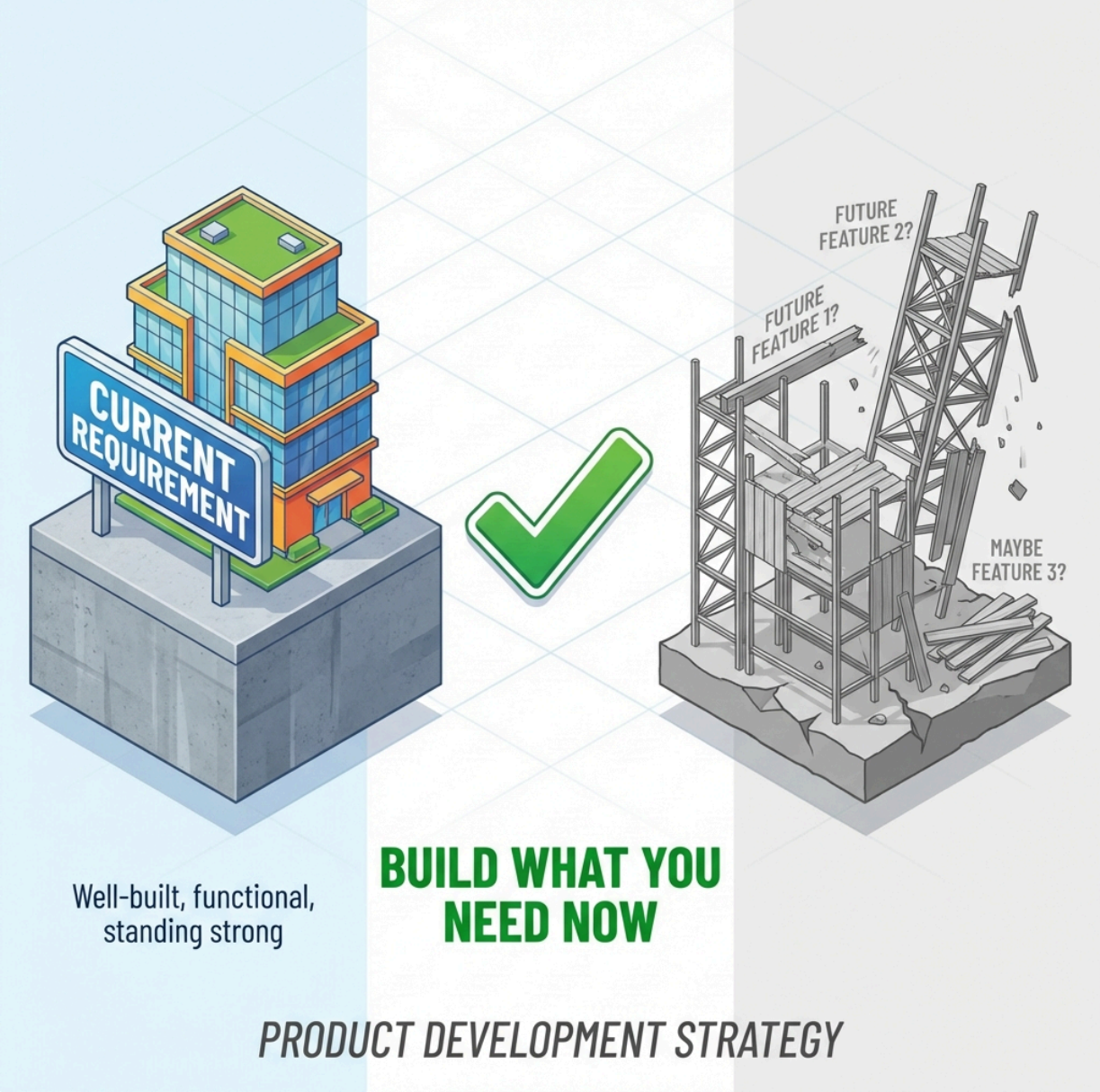
```
// Tránh if-else lồng nhau > 3 cấp
```


YAGNI - You Aren't Gonna Need It

Don't implement something until it is necessary - *Không cần thì đừng làm*

Giải thích

- ☕ Chỉ implement những gì CẦN THIẾT hiện tại
- ☕ Không code cho **tương lai có thể dùng**
- ☕ Tránh **over-engineering**



Eg: Violate YAGNI

```
// Requirement: Lưu user vào MySQL
public interface UserRepository {
    void save(User user);

    // "Tương lai có thể cần"
    void saveToMongoDB(User user);
    void saveToRedis(User user);
    void saveToElasticsearch(User user);
    void export ToJSON(User user);
    void exportToXML(User user);
    void exportToCSV(User user);

    // "Có thể cần cho analytics"
    void saveUserActivity(User user);
    void trackUserBehavior(User user);
}
```

```
public class MySQLUserRepository implements UserRepository {
    public void save(User user) { /* Implementation */ }

    // Phải implement TẤT CẢ method không cần thiết!
    public void saveToMongoDB(User user) {
        throw new UnsupportedOperationException("Not needed yet");
    }
    // ... 7 methods nữa không dùng đến
}
```

- ☕ Code **phức tạp** không cần thiết
- ☕ **Tổn thời gian** implement
- ☕ Khó maintain
- ☕ Có thể không bao giờ dùng đến

Eg: Follow YAGNI

```
// Chỉ implement những gì CẦN
public interface UserRepository {
    void save(User user); // Chỉ cần MySQL hiện tại
}

public class MySQLUserRepository implements UserRepository {
    public void save(User user) {
        // MySQL implementation
    }
}

// Khi THỰC SỰ cần MongoDB → tạo mới
// public class MongoDBUserRepository implements UserRepository { ... }

// Khi THỰC SỰ cần export → tạo service mới
// public class UserExportService { ... }
```

Lợi ích

- ☕ Code **đơn giản** hơn
- ☕ Phát triển **nhANH** hơn
- ☕ Dễ **maintain** hơn
- ☕ Tập trung vào **requirement thực tế**

YAGNI nói KHÔNG với

- ☕ Code cho **có thể cần**
- ☕ Features không ai yêu cầu
- ☕ Optimization sớm
- ☕ Abstraction phức tạp không cần thiết

YAGNI KHÔNG nói KHÔNG với

- ☕ Clean code practices
- ☕ **SOLID** principles
- ☕ Design patterns
- ☕ Testing

DRY vs KISS vs YAGNI

Nguyên lý	Mục đích	Khi nào dùng
DRY	Tránh lặp code	Logic giống nhau ở nhiều nơi
KISS	Giữ đơn giản	Thiết kế giải pháp
YAGNI	Không code thừa	Quyết định implement feature

Eg: DRY ft. KISS ft. YAGNI

```
// YAGNI: Chỉ cần tính tổng (không cần trung bình, tích, ...)  
// KISS: Dùng for loop đơn giản (không dùng Stream phức tạp)  
// DRY: Extract thành method tái sử dụng
```

```
public class Calculator {  
    // Simple, needed, reusable  
    public int sum(int[] numbers) {  
        int total = 0;  
        for (int num : numbers) {  
            total += num;  
        }  
        return total;  
    }  
}
```

Best Practices

Trước khi implement

-  [] Requirement rõ ràng chưa? (**YAGNI**)
-  [] Giải pháp đơn giản nhất? (**KISS**)
-  [] Có cần abstraction không? (**OCP**, **DIP**)

Khi implement

-  [] Mỗi class làm 1 việc? (**SRP**)
-  [] Có logic lặp không? (**DRY**)
-  [] Interface có quá lớn không? (**ISP**)

Sau khi implement

-  [] Code dễ hiểu không? (**KISS**)
-  [] Thay thế được không? (**LSP**)
-  [] Coupling thấp không? (**DIP**)

Code Smell	Nguyên lý vi phạm	Giải pháp
God Class	SRP	Tách thành nhiều class
Duplicate Code	DRY	Extract method/class
Long Method	SRP, KISS	Tách nhỏ method
Switch Statements	OCP	Dùng polymorphism
Tight Coupling	DIP	Dùng interface
Fat Interface	ISP	Tách interface
Dead Code	YAGNI	Xóa bỏ

Problem

Scenario: Hệ thống quản lý đơn hàng vi phạm những nguyên lý nào? Gợi ý cách điều chỉnh?

```
public class OrderManager {  
    public void processOrder(Order order) {  
        // Validate  
        if (order.items.isEmpty()) throw new Exception("Empty order");  
  
        // Calculate  
        double total = 0;  
        for (Item item : order.items) {  
            total += item.price * item.quantity;  
        }  
  
        // Apply discount  
        if (order.customer.type.equals("VIP")) total *= 0.9;  
        else if (order.customer.type.equals("MEMBER")) total *= 0.95;  
    }  
}
```

```
// Payment  
if (order.paymentType.equals("CARD")) {  
    // Process card  
} else if (order.paymentType.equals("CASH")) {  
    // Process cash  
}  
  
// Save  
connection.execute("INSERT INTO orders...");  
  
// Send email  
smtp.send(order.customer.email, "Order confirmed");  
}
```

Solution

Refactor theo SOLID

```
// SRP: Tách trách nhiệm
public class OrderValidator { ... }
public class PriceCalculator { ... }
public class DiscountService { ... }

// OCP: Mở rộng dễ dàng
public interface PaymentMethod { void process(double amount); }
public class CardPayment implements PaymentMethod { ... }
public class CashPayment implements PaymentMethod { ... }

// DIP: Phụ thuộc abstraction
public interface OrderRepository { void save(Order order); }
public interface EmailService { void send(String to, String msg); }

// Main service
public class OrderService {
    private OrderValidator validator;
```

```
    private PriceCalculator calculator;
    private DiscountService discountService;
    private OrderRepository repository;
    private EmailService emailService;

    public void processOrder(Order order, PaymentMethod payment) {
        validator.validate(order);
        double total = calculator.calculate(order);
        total = discountService.apply(total, order.getCustomer());
        payment.process(total);
        repository.save(order);
        emailService.send(order.getCustomer().getEmail(), "Order confirmed");
    }
}
```

Anti-Patterns

Over-Engineering

```
// Không cần: Interface cho class chỉ có 1 implementation
public interface UserService { ... }
public class UserServiceImpl implements UserService { ... }
```

Under-Engineering

```
// Quá đơn giản: Không có abstraction khi cần
public class OrderService {
    private MySQLDatabase db = new MySQLDatabase(); // Tight coupling
}
```


Anti-Patterns (cont)

Premature Abstraction

```
// Tạo abstraction trước khi biết pattern  
public interface Strategy { ... } // Chưa biết cần strategy pattern chưa
```

Make it **work**, make it **right**, make it **fast** - Kent Beck

Tools & Metrics

Static Analysis

- ☕ **SonarQube**: Phát hiện code smells, duplications
- ☕ **CheckStyle**: Kiểm tra coding standards
- ☕ **PMD**: Tìm potential bugs

Code Coverage

- ☕ **Test coverage**: nên > 80%
- ☕ **Lưu ý**: coverage cao ≠ code tốt!

Metrics

- ☕ **Cyclomatic Complexity**: Độ phức tạp (nên < 10)
- ☕ **Coupling**: Số phụ thuộc giữa classes
- ☕ **Cohesion**: LCOM (Lack of Cohesion of Methods)

Evolution of Design

- ☕ Design tốt không đến ngay từ đầu
- ☕ **Refactor** thường xuyên
- ☕ **Refactor** từng bước nhỏ
- ☕ **Test** trước khi refactor



Common Questions

Q: Khi nào nên vi phạm SOLID?

A: Khi tuân thủ làm code phức tạp hơn không cần thiết. Principles là guidelines, không phải laws.

Q: DRY có mâu thuẫn với KISS không?

A: Không. DRY loại bỏ duplicate, KISS giữ logic đơn giản. Cả hai hướng tới code dễ maintain.

Q: Làm sao biết khi nào đủ abstraction?

A: "Rule of Three" - Khi thấy duplicate lần thứ 3, hãy abstract. Lần 1-2 có thể còn sớm.

Q: YAGNI có nghĩa là không cần design?

A: Không. YAGNI là không implement feature chưa cần, nhưng vẫn cần good design cho code hiện tại.

Mindset Shift

Chuyển từ

- ☕ "Code chạy được là đủ"
- ☕ "Viết nhanh rồi sửa sau"
- ☕ "Chỉ mình tôi đọc code này"

Sang

- ☕ "Code phải dễ maintain"
- ☕ "Viết đúng từ đầu, refactor thường xuyên"
- ☕ "Code cho team và future self"

"Any fool can write code that a computer can understand. Good programmers write code that humans can understand." - *Martin Fowler*

Q&A

CẢM ƠN ĐÃ THEO DÕI